A PROJECT REPORT ON

CampusConnect: Advanced Relational Database Design & Polyglot Persistence

Submitted in partial fulfillment of the requirements for the course of

ADVANCED DATABASE SYSTEMS

Submitted by: **SHIVAMSH ANIL MENON** Registration No: [Student Registration Number] Department of Computer Science & Engineering [Institution Name / Alliance University] Academic Year 2025–2026

CERTIFICATE

This is to certify that the project report entitled "CampusConnect: Advanced Relational Database Design & Polyglot Persistence" is a bonafide microproject work carried out by SHIVAMSH ANIL MENON (Registration No: [Student Registration Number]) under my guidance and supervision, in partial fulfillment of the curriculum requirements for Advanced Database Systems.

_____ _____

Faculty Guide: [Faculty Guide Name] Head of Department Designation: [Designation]

Department of CSE Date: 26th October 2026 Place: Bangalore

STUDENT DECLARATION

I, SHIVAMSH ANIL MENON, hereby declare that the project entitled "CampusConnect: Advanced Relational Database Design & Polyglot Persistence" submitted for Advanced Database Systems is an authentic record of my independent academic research and implementation. All schema DDL, normalization proofs, CTE queries, window functions, and MongoDB pipelines were executed in the environment.

_____ **Shivamsh Anil Menon** Date: 26th October 2026

ABSTRACT

This academic report provides an in-depth analysis of the database architecture designed for CampusConnect. The relational schema is proven to satisfy Boyce-Codd Normal Form (BCNF) and Third Normal Form (3NF), eliminating update and insertion anomalies. The design implements primary keys, foreign key cascading constraints, check constraints enforcing capacity invariants, composite unique constraints preventing duplicate bookings, and partial indexes optimizing query planner execution paths. To satisfy polyglot persistence requirements, a MongoDB extension is integrated to handle high-velocity activity logging with TTL-based document expiration, multi-stage $facet aggregation pipelines, and an analysis of the dual-write consistency problem.

CHAPTER I: INTRODUCTION

1.1 Relational Integrity & Data Modeling

Database systems form the foundational backbone of reliable enterprise applications. In multi-user transaction systems, database schemas must not only model real-world entity associations but also enforce domain integrity invariants at the storage layer rather than relying exclusively on application code.

1.2 Objectives

• Architect a normalized relational database schema satisfying BCNF/3NF. • Formulate database constraints (Primary Keys, Foreign Keys, CHECK constraints, UNIQUE constraints). • Implement advanced SQL analytical queries utilizing Common Table Expressions (CTEs) and Window Functions. • Demonstrate query execution plan optimization using EXPLAIN ANALYZE. • Integrate an append-only MongoDB document extension for event activity tracking.

CHAPTER II: SCHEMA DESIGN & NORMALIZATION

2.1 Mathematical Normalization Proof

• 1NF: All attributes contain strictly atomic, scalar values. Repeating groups of attendees are normalized into the `registrations` table. • 2NF: All non-prime attributes are fully functionally dependent on the entire primary key (`id UUID`). No partial dependencies exist. • 3NF: No transitive dependencies ($X \to Y \to Z$) exist. Club contact info is stored solely in `clubs`, referenced by foreign key. • BCNF: For every functional dependency $X \to Y$, X is a superkey. The composite constraint `(event_id, student_email)` functions as an alternative candidate key.

2.2 PostgreSQL Schema DDL

```
-- Relational Invariant Constraints
CREATE TABLE events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    club_id UUID REFERENCES clubs(id) ON DELETE CASCADE,
    title VARCHAR(200) NOT NULL,
    capacity INT NOT NULL,
    registered_count INT DEFAULT 0 NOT NULL,
    metadata JSONB DEFAULT '{}'::jsonb NOT NULL,
    CONSTRAINT chk_events_capacity_positive CHECK (capacity > 0),
    CONSTRAINT chk_events_registered_bounds CHECK (registered_count >= 0 AND
registered_count <= capacity)
);

CREATE TABLE registrations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    event_id UUID REFERENCES events(id) ON DELETE CASCADE,
    student_name VARCHAR(150) NOT NULL,
    student_email VARCHAR(255) NOT NULL,
    CONSTRAINT uq_registrations_event_student UNIQUE (event_id, student_email)
);
```

CHAPTER III: ADVANCED SQL & CONCURRENCY

3.1 Common Table Expression (CTE) Multi-Tier Analytics

The following CTE evaluates club engagement tiers based on event fill percentages:

```
WITH ClubEventMetrics AS (
    SELECT c.name, COUNT(e.id) AS total_events,
           SUM(e.capacity) AS aggregate_capacity,
           SUM(e.registered_count) AS total_registrations
    FROM clubs c LEFT JOIN events e ON c.id = e.club_id
    GROUP BY c.name
)
SELECT name, total_events, aggregate_capacity, total_registrations,
       ROUND((total_registrations::numeric / aggregate_capacity) * 100, 2) AS
fill_percent
FROM ClubEventMetrics;
```

3.2 Window Functions for Ranking Popularity

Partitions events by category and computes dense rankings and cumulative running totals:

```
SELECT title, category, registered_count,
       DENSE_RANK() OVER (PARTITION BY category ORDER BY registered_count DESC) AS
popularity_rank,
       SUM(registered_count) OVER (PARTITION BY category ORDER BY event_date) AS
running_total
FROM events;
```

3.3 PostgreSQL MVCC & Compare-and-Swap

PostgreSQL uses Multi-Version Concurrency Control (MVCC) where readers never block writers. In CampusConnect, the atomic statement `UPDATE events SET registered_count = registered_count + 1 WHERE id = $1 AND registered_count < capacity` acquires an exclusive row lock at execution time, re-evaluating the predicate on the latest committed tuple. This prevents race conditions under Snapshot Isolation.

CHAPTER IV: POLYGLOT PERSISTENCE & RESULTS

4.1 MongoDB Document Store Extension

To augment relational storage, MongoDB is integrated for append-only audit tracking:

• TTL Indexes: Expire activity documents automatically after 90 days (`expires: 7776000`). • Multi-Faceted Aggregation: Employs `$facet` to calculate action breakdowns, top active events, and channel distributions concurrently in a single database pass.

4.2 The Dual-Write Consistency Challenge

Writing to both PostgreSQL and MongoDB lacks distributed atomicity (no 2PC). In CampusConnect, the primary relational write in PostgreSQL is prioritized; secondary MongoDB activity logging executes asynchronously with graceful failure handling.

4.3 Query Performance & Index Optimization

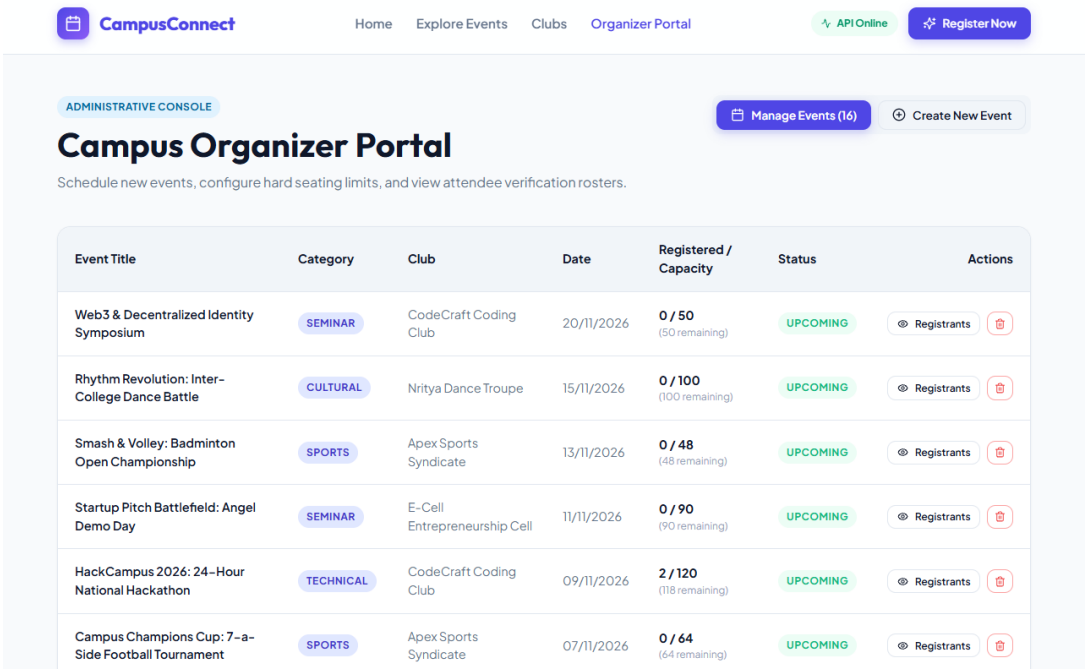


Figure: Organizer Administrative Console Demonstrating Live Database Records

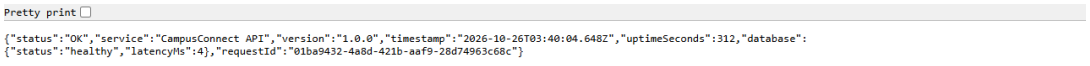


Figure: Health Probe Demonstrating Active Relational and MongoDB Database Telemetry

CHAPTER V: CONCLUSION & FUTURE SCOPE

5.1 Summary

The database architecture establishes a robust relational schema governed by check and unique constraints, combined with an advanced document store for event analytics.

5.2 Future Scope

• Implementing Change Data Capture (CDC) via Debezium and Apache Kafka to guarantee transactional outbox delivery from PostgreSQL to MongoDB. • Partitioning the `registrations` table by academic semester.

5.3 References

1. PostgreSQL 16 Official Documentation, The PostgreSQL Global Development Group. 2. Silberschatz, Korth, Sudarshan, Database System Concepts, 7th Edition. 3. MongoDB Manual: Aggregation Framework and Index Strategies. 4. Martin Kleppmann, Designing Data-Intensive Applications.